

README

Part 15: 多模态理解（VLM） — 视觉如何"说"语言

- > 让模型"看懂"图片并对话——这是 2026 年所有旗舰模型的标配（GLM-5.3-Flash、Qwen3-VL 均为原生多模态）。本章手写拼接式 VLM 的四件套（LLaVA 架构的最小闭环），再横向对比三大主流方案与对齐损失。学完你能独立设计/调试一个多模态系统。
- > 锚点仓库：[huggingface/nanoVLM](https://github.com/huggingface/nanoVLM) (5.0k, "VLM 版 nanoGPT")
- > · [jingyaogong/minimind-v](https://github.com/jingyaogong/minimind-v) (8.5k, 中文 65M)
- > · 工业对照 [Qwen3-VL](https://github.com/QwenLM/Qwen3-VL) (19.9k)

学习目标

完成本部分后，你将能够：

- 理解 多模态理解在 LLM 链路中的位置和价值
- 手写 拼接式 VLM 的四件套（Patch Embedding + ViT + Projector + Token 拼接）
- 解释 CLIP/SigLIP 的数学原理、对齐损失与 batch 依赖性差异
- 画出 三大方案（拼接式/门控/early-fusion）的注入位置图并完成选型
- 识别 多模态理解中的常见陷阱（静态图缓存、温度 τ 、batch 依赖）并设计防范策略

章节导航

章节	内容	对应
[手写拼接式 VLM 四件套](01_handwritten_projection_vlm.md)	patch embedding → ViT → mlp2x 投影器 → token 拼接；LLaVA 两阶段训练玩具版	'01'
[三大方案与对齐损失](02_alignment_losses_and_schemes.md)	拼接式/门控/early-fusion；CLIP InfoNCE vs SigLIP；动态分辨率与 token 压缩	'02'

理论背景（导览）

- 为什么需要多模态：纯文本模型"只能听"，跨模态对齐让它"又能看"——完整的痛点引入与类比在 [01 章](01_handwritten_projection_vlm.md)。
- 两条数学主线：拼接式数据流（Patch → ViT → Projector → 拼接，含逐步形状账本）在 [01 章](01_handwritten_projection_vlm.md)；对齐损失推导（CLIP InfoNCE softmax vs SigLIP 逐对 sigmoid）在 [02 章](02_alignment_losses_and_schemes.md)。
- 一句话洞察：多模态的门槛不在"造模型"，在"训对齐"——LLaVA 的全部增量只是一个 2 层 MLP 投影器（`mlp2x_gelu`）+ 两阶段训练；对比学习中 temperature 控制相似度分布的锐度，batch 越大负样本越多、对比信号越强。

前置知识

必须掌握：

- [Part 6 03 章 Transformer Block](../Part6_transformer/tutorial/03_transformer_block.md)：本章的 ViTBlock 与玩具 LLM 都是它的同款（残差 + pre-norm + 注意力），看懂它 = 看懂半个 ViT

- [Part 8 02 章 SFT 与对话](../Part8_post_training/tutorial/02_sft_and_chat.md)：prompt masking (labels 置 -100) ——两阶段训练只监督 answer 段，全靠它

建议掌握：

- [Part 8 07 章评估](../Part8_post_training/tutorial/07_evaluation.md)："规则评估 vs 学习评估"的对比思维，直接迁移到 CLIP/SigLIP 的对比损失 (02 章)
- [Part 8 08 章 LoRA](../Part8_post_training/tutorial/08_lora_and_classification.md)：资源紧张时 Stage 2 用 LoRA 代替全参微调 (对照投影器"小参数撬动大模型"的账)

可选：

- 零视觉基础完全可学：patch embedding 就是"切块 + 线性投影"，ViT 块就是 Part 6 的 Block——本部分不要求任何 CV 前置

在 LLM 链路中的位置

【本部分: 视觉→语言 对齐（理解侧）】→ 与 Part 16 生成侧互为镜像
 图像——ViT——► 视觉 token ——projector——► LLM token 空间 ——► 对话

环境

脚本 01/02 均 CPU 可跑、零新依赖 (torch 即可)；教程中引用的实测数字为 RTX 4090 复跑结果 (已逐项核对一致)，CPU 结果同量级。02 章为"方案对照 + 对齐损失"；工业模型推理实操 (SmolVLM-500M <1.5GB、Qwen2-VL-2B ~5GB, 4090 全兼容) 为进阶自练，权重获取见 [docs/datasets.md § 5](../docs/datasets.md)。

学习地图

手写四件套 (脚本01：形状账本) ← 点
 ↓ "我的 mlp2x_gelu 就是 LLaVA 的同名结构"
 两阶段训练 (Stage1 对齐/Stage2 指令)
 ↓
 三大方案对照 + CLIP vs SigLIP ← 面 → 面试/选型就绪

课后作业

[Assignment 15](../assignments/assignment_15/)

相关资源

- [nanoVLM](https://github.com/huggingface/nanoVLM) (~750 行纯 PyTorch VLM, SigLIP+SmolLM2=222M)

- [minimind-v](https://github.com/jingyaogong/minimind-v) (中文 65M, 3090 数小时可训, 两阶段与 LLaVA 论文一一对应)
- [LLaVA](https://github.com/haotian-liu/LLaVA) (25k, 注意仓库名; 续作 LLaVA-NeXT)
- CLIP (2103.00020) · SigLIP (2303.15343) · LLaVA (2304.08485) · Qwen2-VL (2409.12191) · Flamingo (2204.14198)

[← 上一章: Part 14 vLLM](../Part14_inference_vllm/tutorial/README.md) | [下一章: Part 16 图像/视频生成] (../Part16_image_video_generation/tutorial/README.md)

01_handwritten_projection_vlm

01 — 手写拼接式 VLM 四件套

> 拼接式 (projector) 方案占了 2026 年开源 VLM 的绝大多数 (LLaVA/SmolVLM/Qwen-VL/InternVL 全是这个家族)。它的全部秘密只有一句话: **把图像变成一串"LLM 能消费的 token", 拼进文本序列**。本章手写这四件套 (跑 [scripts/01_vit_projector_pipeline.py](../scripts/01_vit_projector_pipeline.py), CPU 10 秒), > 并跑 LLaVA 两阶段训练的玩具版。

学习目标

完成本章后, 你将能够:

- 手写 拼接式 VLM 的四件套 (Patch Embedding + ViT + Projector + Token 拼接)
- 解释 Projector 的角色 (模态翻译器) 与两阶段训练的理由
- 画出 拼接式 VLM 的数据流并标注每步 shape (形状账本)
- 识别 "静态图缓存 backward" 等三个真实实现坑

前置知识

必须掌握:

- Part 6: Transformer block (本章 ViTBlock 同款)
- Part 8 02 章: prompt masking

理论背景

问题引入: 为什么需要多模态理解?

预训练模型虽然强大, 但只能处理文本:

1. 模态限制: 无法理解图像、视频、音频等非文本信息
2. 任务限制: 无法完成图像描述、视觉问答等多模态任务

多模态理解通过跨模态对齐来弥补:

纯文本模型: "只能处理文本"

多模态模型: "可以理解图像、视频、音频"

> 类比：纯文本模型像是只会听的人，多模态模型像是会听又会看的人。看的能力让理解更全面。

数学推导：Projector 的作用

问题设定：

- 视觉特征： $v \in \mathbb{R}^{d_v}$ （来自 ViT）
- LLM 维度： d_l （LLM 的 embedding 维度）
- Projector：把 d_v 维映射到 d_l 维

推导过程：

Step 1: 视觉编码

$v = \text{ViT}(\text{image}) \# (B, n_{\text{patches}}, d_v)$

Step 2: 投影

$v_{\text{proj}} = \text{Projector}(v) \# (B, n_{\text{patches}}, d_l)$

$\text{Projector} = \text{Linear}(d_v, d_l) \rightarrow \text{GELU} \rightarrow \text{Linear}(d_l, d_l)$

Step 3: 拼接

$\text{input} = [v_{\text{proj}}; \text{text_embed}] \# (B, n_{\text{patches}} + n_{\text{text}}, d_l)$

Step 4: LLM 处理

$\text{output} = \text{LLM}(\text{input}) \# (B, n_{\text{patches}} + n_{\text{text}}, d_l)$

关键洞察：

- Projector 是"模态翻译器"，把视觉特征翻译成 LLM 能理解的语言
- LLM 本体不感知模态差异，只看到 embedding
- 两阶段训练：先训 Projector（搭桥），再端到端微调（稳固）

代码实现

1. 四件套的"形状账本"

运行 [scripts/01_vit_projector_pipeline.py](../scripts/01_vit_projector_pipeline.py) 验证以下代码。

图像 (B,3,8,8)

- ① $\text{PatchEmbed}(\text{Conv } k=s=2) \rightarrow (B, 16, 24) \# (8/2)^2 = 16 \text{ 个视觉 token, 每个 } 24 \text{ 维}$
- ② $\text{ViTBlock} \times 2 \rightarrow (B, 16, 24) \# \text{ 视觉 token 之间先"自交流" (双向注意力)}$
- ③ $\text{Projector mlp2x_gelu} \rightarrow (B, 16, 32) \# \text{ 翻译成 LLM 维度} \leftarrow \text{"翻译器"}$
- ④ $\text{token 拼接} \rightarrow (B, 16 + \text{文本}, 32) \# \text{ 图像向量直接当 LLM 输入 embedding}$

形状追踪：拼接式 VLM 数据流

工程实践

调试展示：常见错误与修复

错误 1：静态图缓存 backward

症状：

```
RuntimeError: Trying to backward through the graph a second time
```

原因：X 的计算图含可训练投影器，必须每步重建

解法：

```
python
```

每步重建计算图

```
for step in range(num_steps):  
    X = model(input) # 重新计算，不要缓存  
    loss = criterion(X, target)  
    loss.backward()  
python
```

错误 2：图像位置查 embedding 表

症状：

```
IndexError: index out of range
```

原因：图像位置不能查 embedding 表，输入就是投影向量

解法：

```
python
```

图像位置直接用投影向量，不查 embedding 表

```
input_embed[:, :n_image, :] = visual_tokens # 图像 token  
input_embed[:, n_image:, :] = text_embed[text_ids] # 文本 token  
python
```

错误 3：device 不一致

症状：

```
RuntimeError: Expected all tensors to be on the same device
```

原因：文本 id 张量的 device 要与 tok_weight 对齐

解法：

```
`python
```

确保所有张量在同一设备

```
text_ids = text_ids.to(device)
visual_tokens = visual_tokens.to(device)
```

性能数据（实测参考）

模型	方法	可训练参数	训练时间	效果
本脚本玩具版（200+100 步）	Stage 1	1,856	<1min（4090 实测 ~2s）	loss 2.907→1.825
本脚本玩具版（200+100 步）	Stage 2	29,308	<1min（4090 实测 ~2s）	loss→0.034
7B	LLaVA Stage 1	~20M	~2h	对齐视觉特征
7B	LLaVA Stage 2	~7B	~10h	指令微调

> 数据来源：LLaVA 论文 + 本课开发机实测

常见陷阱

陷阱 1：两阶段训练顺序错误

症状：效果不好，或训练不稳定

原因：直接端到端微调，没有先训 Projector

解法：先 Stage 1（只训 Projector），再 Stage 2（端到端微调）

陷阱 2：图像分辨率不匹配

症状：token 数不对，或效果不好

原因：图像分辨率与模型期望不匹配

解法：检查模型期望的分辨率（如 LLaVA 用 336^2 ）

陷阱 3：prompt 格式不对

症状：模型不理解图像

原因：prompt 格式与模型的 chat template 不匹配

解法：检查 prompt 格式（如 `<image>\n问题`）

最佳实践

两阶段训练配置

阶段	可训练模块	学习率	数据量	说明
Stage 1	Projector	1e-3	558K	特征对齐
Stage 2	全部	2e-5	665K	指令微调

Prompt 格式

```
`python
LLaVA 格式
```

```
prompt = "<image>\n" + question
```

Qwen-VL 格式

```
prompt = "<|vision_start|><|image_pad|><|vision_end|>" + question`
```

学完本章你能...

- 画出拼接式 VLM 的数据流并标注每步 shape（形状账本）
- 说清 projector 的角色（模态翻译器）与两阶段训练的理由
- 指出 LLaVA→Qwen-VL 的关键演进（固定分辨率 → 原生动态分辨率）
- 绕开"静态图缓存 backward"等三个真实实现坑

概念检验

```
<details>
<summary>Q1: Stage 1 如果不冻结 ViT 和 LLM 会怎样？</summary>
```

A: 投影器随机初始化时输出是"噪声 token"，两端未对齐——同时更新三个模块，LLM 会被噪声 token 冲得偏离预训练分布（灾难性遗忘），ViT 也偏离其 CLIP 对齐。先训投影器 = 在固定的两端之间"搭桥"，桥稳了再动两端。

```
</details>
<details>
<summary>Q2: 16 个视觉 token 拼进序列后，LLM 的注意力为什么用因果遮罩也合理？</summary>
```

A: 图像 token 在序列最前，文本 token 在后——因果遮罩下文本能看到全部图像 token（这正是"看图"），图像 token 之间互相可见（位置 0-15 互在前缀内）。若把图像放中间则需要分段遮罩（图像段双向、文本段因果）——部分实现确实这么做。

```
</details>
<details>
<summary>Q3: Projector 为什么用 MLP 而不是线性层？</summary>
```

A: MLP（Linear→GELU→Linear）有非线性变换，表达能力更强。线性层只能做线性映射，无法处理复杂的模态转换。经验上 MLP 通常比线性层效果更好（属经验值，具体幅度因任务而异）。

```
</details>
```

动手实践

```
<details>
```

<summary>练习 1: 实现 Patch Embedding</summary>

任务：实现一个函数，把图像转换成 patch embedding。

验收标准：

- [] 输入：图像 (B, C, H, W)
- [] 输出：patch embedding (B, n_patches, d_model)
- [] 使用 Conv2d 实现

步骤提示：

```
`python
def patch_embed(image, patch_size=2, d_model=24):
    """
```

Steps:

1. 使用 Conv2d 提取 patch
2. 重塑为 (B, n_patches, d_model)
3. 返回 patch embedding

```
"""
```

TODO: Implement

```
pass
`
```

</details>

<details>

<summary>练习 2: 实现 Projector</summary>

任务：实现一个函数，把视觉特征投影到 LLM 维度。

验收标准：

- [] 输入：视觉特征 (B, n_patches, d_v)
- [] 输出：投影特征 (B, n_patches, d_l)
- [] 使用 MLP (Linear→GELU→Linear)

步骤提示：

```
`python
def projector(visual_feat, d_v=24, d_l=32):
    """
```

Steps:

1. Linear(d_v, d_l * 2)
2. GELU
3. Linear(d_l * 2, d_l)
4. 返回投影特征

```
"""
```

TODO: Implement

```
pass
`
```

</details>

<details>

<summary>练习 3: 实现 token 拼接</summary>

任务：实现一个函数，把视觉 token 和文本 embedding 拼接。

验收标准：

- [] 输入：visual_tokens (B, n_v, d_l), text_embed (B, n_t, d_l)
- [] 输出：拼接后的 embedding (B, n_v + n_t, d_l)
- [] 正确处理维度

步骤提示：

```
python
def concat_tokens(visual_tokens, text_embed):
    """
```

Steps:

1. 检查维度是否匹配
 2. 使用 torch.cat 拼接
 3. 返回拼接后的 embedding
- ```
"""
```

## TODO: Implement

```
pass
```

</details>

## 课后作业

完成本章后，去 Assignment 15 完成练习：

[Assignment 15](../../assignments/assignment\_15/)

## 下一步

会"用"图像了。但视觉特征和文本特征在同一个空间里对齐是更底层的能力（检索、打分、引导生成全靠它）——以及三大方案的全景对比。

[02 — 三大方案与对齐损失](02\_alignment\_losses\_and\_schemes.md)

## 02\_alignment\_losses\_and\_schemes

### 02 — 三大方案与对齐损失（CLIP vs SigLIP）

- > 01 章手写了"拼接式"这一主流方案。本章把视野拉开：① 三大架构方案的对照；
- > ② 对齐的底层损失——CLIP 的 InfoNCE 与 SigLIP 的 sigmoid 成对损失（跑
- > [scripts/02\_clip\_siglip\_alignment.py](../scripts/02\_clip\_siglip\_alignment.py)，CPU 10 秒，
- > 两种损失都收敛且检索 100%）。

## 学习目标

完成本章后，你将能够：

- 画出 三大方案（拼接式/门控/early-fusion）的注入位置图并说出代表模型与现状
- 实现 InfoNCE 与 SigLIP 两种对齐损失（对称双方向 softmax CE / 逐对 sigmoid）
- 解释 两者的 batch 依赖性差异与温度  $\tau$  的作用（可学习、控锐度）
- 区分 对比式对齐与生成式对齐的适用场景（检索/打分 vs 让 LLM 消费视觉 token）
- 估算 动态分辨率与 token 压缩下的视觉 token 数（Qwen-VL 式预算控制）

## 前置知识

- 01 章：拼接式四件套；Part 8 07 章：对比"规则评估 vs 学习评估"的思维
- 概率论：softmax 与 sigmoid 的关系（前者是后者的全局归一化版）

## 1. 三大方案全景（2026 开源格局）

|                                         | 注入机制                                               | 优/劣                |
|-----------------------------------------|----------------------------------------------------|--------------------|
| Qwen-VL、InternVL、nanoVLM、minimind-v     | 视觉 token **拼进序列**，LLM 无改动                          | 简单、复用全部 LLM 生态     |
|                                         | Perceiver Resampler 压缩视觉 → **gated xattn** 注入冻结 LM | 可保留纯文本能力；结构复杂      |
| 投影直入 LLM，无视觉编码器）、Chameleon（VQ 图像 token） | 图像就是"另一种 token"                                    | 训练贵但上限高；any-to-any |

- 一条演进主线：拼接式（外挂翻译器）→ 原生多模态（预训练时就混模态）。GLM-5.3-Flash（GLM 系首个原生多模态）、DeepSeek-V4 的混合模态注意力，都是 (c) 路线的工业兑现。理解 (a) 是理解 (c) 的前提——注入点从"输入侧"移到"预训练数据侧"。
- 细节差异点（面试常问）：Qwen-VL 系的原生动态分辨率——不把图压到固定  $336^2$ ，而是按原始尺寸切 patch 打包（token 数随内容变），OCR/图表类任务大幅受益；InternVL 的像素洗牌——把相邻  $2 \times 2$  的视觉通道重排进特征维，视觉 token 数直接  $\div 4$ 。

## 2. 对齐损失：CLIP InfoNCE vs SigLIP（跑脚本 02）

玩具实验：4 个概念，图像/文本各有一个塔投影到共享空间，两种损失训练后图→文检索 top-1 全部 100%——两种方法都能对齐，但行为不同：

```
python
```

InfoNCE（CLIP）： $N \times N$  相似度矩阵按行/列 softmax，标签=对角线

```
logits = scale * f_img @ f_txt.T # scale = 可学习温度 τ
loss = 0.5 * (CE(logits, labels) + CE(logits.T, labels)) # 对称双方向
```

SigLIP：逐对 sigmoid（对角 +1，非对角 -1），无全局归一化

```
targets = 2 * eye(N) - 1
loss = -F.logsigmoid(targets * logits).mean()
```

|          | InfoNCE (CLIP, 2103.00020)      | SigLIP (2303.15343)               |
|----------|---------------------------------|-----------------------------------|
| 归一化      | 行/列 softmax (全 batch 参与)        | 逐对独立 sigmoid                      |
| batch 依赖 | **强** (负例来自 batch, 小 batch 信号弱) | 弱 (论文实测 batch 1/4 持平)             |
| 使用者      | CLIP、LLaVA 的视觉塔                 | SigLIP、SmolVLM、InternVL、PaliGemma |

- 温度  $\tau$  的作用： $\text{scale} = \exp(\log\_scale)$  可学习——控制 softmax 锐度。  
 $\tau$  太小对比信号弱，太大早期训练不稳。脚本实测（RTX 4090 复跑）学习到的  $\tau$ ：CLIP 路径 16.21，SigLIP 8.53。
- 和生成侧的连接（Part 16 的地基）：对比对齐学到的共享空间，正是生成模型 cross-attention 消费的空间——文本嵌入能"指挥"图像生成，前提是两个模态在这个空间里已经对齐。理解侧（本章）与生成侧（Part 16）共享同一个对齐世界观。

### 3. 两阶段与对齐损失的关系

LLaVA Stage 1 用的是生成式对齐（图文对上的 next-token loss 只训投影器），而不是 CLIP 式对比对齐——为什么？因为投影器的目标不是"检索"，而是"让 LLM 读得懂"。  
两条对齐路线：

对比式（CLIP/SigLIP）：拉近配对、推远非配对 → 适合检索/打分/视觉塔预训练  
生成式（LLaVA Stage1）：图文对上的 CE → 适合"让 LLM 消费视觉 token"  
现代实践：视觉塔用 CLIP/SigLIP 预训练好，Stage 1 再做生成式投影对齐——两条都用

### 工程实践

#### 调试展示：常见错误与修复

错误 1：温度  $\tau$  初始化不当

症状：

loss 不下降，或训练不稳定

原因：温度  $\tau$  太小（对比信号弱）或太大（早期训练不稳）

解法：

```
python
```

#### 使用可学习的温度参数

```
log_scale = nn.Parameter(torch.log(torch.tensor(1.0 / 0.07))) # CLIP 默认
scale = torch.exp(log_scale)
```

#### 或使用固定温度

```
scale = 1.0 / 0.07 # CLIP 默认
```

```
`
错误 2：batch 太小导致对比学习失败
```

```
`
症状：
```

```
`
loss 不下降，或检索效果差
`
```

原因：batch 太小，负样本太少，对比信号弱

解法：

```
`python
```

## 增大 batch size

```
batch_size = 256 # CLIP 论文用 32768
```

## 或使用梯度累积

```
for i, (images, texts) in enumerate(dataloader):
 loss = criterion(images, texts) / accumulation_steps
 loss.backward()
 if (i + 1) % accumulation_steps == 0:
 optimizer.step()
 optimizer.zero_grad()
`
```

```
`
错误 3：图像和文本维度不匹配
```

```
`
症状：
```

```
`
RuntimeError: mat1 and mat2 shapes cannot be multiplied
`
```

原因：图像特征和文本特征的维度不匹配

解法：

```
`python
```

## 确保维度匹配

```
assert image_feat.shape == text_feat.shape
```

## 或使用投影层对齐维度

```
proj = nn.Linear(image_dim, text_dim)
image_feat = proj(image_feat)
`
```

## 性能数据（实测参考）

| 方法             | batch size | 训练时间          | 检索准确率 | 说明                                        |
|----------------|------------|---------------|-------|-------------------------------------------|
| InfoNCE (CLIP) | 32 (玩具全批)  | <1s (4090 实测) | 100%  | 本课实测：脚本 02, 4 概念 × 8 样本, loss 4.155→1.968 |
| SigLIP         | 32 (玩具全批)  | <1s (4090 实测) | 100%  | 本课实测：脚本 02, 同批数据, loss 0.912→0.106        |
| InfoNCE (CLIP) | 256        | ~1h           | ~95%  | 真实数据 (论文报告值)                              |
| SigLIP         | 64         | ~30min        | ~95%  | 真实数据 (论文报告值)                              |

> 数据来源：前两行为本课实测（脚本 02，开发机复跑）；后两行 ~95% 为论文报告值，非本机复现。

## 常见陷阱

### 陷阱 1：batch 依赖性

症状：小 batch 效果差  
原因：InfoNCE 的负样本来自 batch，batch 小则负样本少  
解法：使用 SigLIP（batch 依赖弱）或增大 batch

### 陷阱 2：温度 $\tau$ 选择不当

症状：训练不稳定，或效果不好  
原因： $\tau$  太小或太大  
解法：使用可学习的温度参数，或从 0.07 开始调整

### 陷阱 3：数据质量问题

症状：效果不好  
原因：图文配对质量差  
解法：使用高质量的图文配对数据

## 最佳实践

### 配置推荐

| 参数            | 推荐值         | 说明               |
|---------------|-------------|------------------|
| batch_size    | 256+        | 越大效果越好           |
| temperature   | 0.07        | CLIP 默认值         |
| learning_rate | 1e-3 ~ 1e-4 | 根据 batch size 调整 |
| epochs        | 10-30       | 根据数据量调整          |

## 学完本章你能...

- 画出三大方案的注入位置图，说出各自代表模型与现状

- 实现 InfoNCE 与 SigLIP，说清 batch 依赖性与温度  $\tau$
- 解释 LLaVA Stage 1 为什么用生成式对齐而视觉塔用对比式预训练
- 估算动态分辨率下图像 token 数（作业题 4）
- 识别温度  $\tau$  初始化、batch 太小等常见陷阱

## 概念检验

<details>

<summary>Q1: 为什么 Flamingo 的 gated xattn 要加一个可学习的门控（tanh 前乘 0 初始化）？</summary>

A: 视觉信息对预训练 LM 是"外语"——门控初始为 0 让视觉分支的扰动从零开始，LM 行为完全不受影响，训练中模型自己决定"开多大门"。这与 LoRA 的  $B=0$ 、ResNet 的零初始化残差是同一个设计模式：新分支从恒等/零出发。

</details>

<details>

<summary>Q2: 一张  $1024 \times 768$  的图，patch 14、压缩率 4（pixel shuffle），大约多少视觉 token？</summary>

A:  $\text{ceil}(1024/14) \times \text{ceil}(768/14) = 74 \times 55 = 4070$  个 patch token,  $\text{pixel shuffle} \div 4 \rightarrow \text{floor}(4070/4) = 1017$  个（与 assignment\_15 题 4 的测试值一致）。

作业题 4 会算：这就是为什么动态分辨率模型要做 token 预算控制（否则长图吃掉整个上下文）。

</details>

<details>

<summary>Q3: 为什么 SigLIP 比 InfoNCE 更适合小 batch？</summary>

A: InfoNCE 的负样本来自 batch，batch 小则负样本少，对比信号弱。SigLIP 使用逐对 sigmoid，不依赖 batch 内的其他样本，因此 batch 依赖性弱。论文实测：batch 1/4 时 SigLIP 与 InfoNCE 持平。

</details>

## 动手实践

<details>

<summary>练习 1: 实现 InfoNCE 损失（与作业题 2、脚本 02 同名同签名）</summary>

任务：实现 CLIP 的 InfoNCE 损失函数。

验收标准：

- [] 输入： $f_{\text{img}}(B, d), f_{\text{txt}}(B, d)$ ，均为归一化特征；scale 为标量（如 10.0）
- [] 输出： $\text{loss}(\text{scalar})$
- [] 使用对称双方向损失（logits 与 logits.T 各做一次 CE）

步骤提示：

```
python
```

```
def infonce_loss(f_img, f_txt, scale):
```

```
"""
```

Steps:

1. 计算相似度矩阵  $\text{logits} = \text{scale} * f_{\text{img}} @ f_{\text{txt}}.T$
2. 创建标签  $\text{labels} = \text{torch.arange}(B)$
3. 计算对称损失  $\text{loss} = 0.5 * (\text{CE}(\text{logits}, \text{labels}) + \text{CE}(\text{logits}.T, \text{labels}))$
4. 返回 loss

```
"""
```

## TODO: Implement

```
pass
```

```
,
```

</details>

<details>

<summary>练习 2: 实现 SigLIP 损失（与脚本 02 同名同签名）</summary>

任务：实现 SigLIP 的 sigmoid 成对损失函数。

验收标准：

- [] 输入：f\_img (B, d), f\_txt (B, d)，均为归一化特征；scale 为标量
- [] 输出：loss (scalar)
- [] 使用逐对 sigmoid（对角 +1，非对角 -1），无全局 softmax

步骤提示：

```
`python
def siglip_loss(f_img, f_txt, scale):
 """
```

Steps:

1. 计算相似度矩阵 logits = scale \* f\_img @ f\_txt.T
2. 创建目标矩阵 targets = 2 \* eye(B) - 1
3. 计算损失 loss = -logsigmoid(targets \* logits).mean()
4. 返回 loss

```
"""
```

## TODO: Implement

```
pass
```

```
`
```

</details>

<details>

<summary>练习 3: 估算动态分辨率 token 数（= 作业题 4，同名同签名）</summary>

任务：实现一个函数，估算 Qwen-VL 式动态分辨率下的视觉 token 数。

验收标准：

- [] 输入：图像高宽 h, w；patch（默认 14）；compress（pixel shuffle 压缩率，默认 4）；max\_tokens 预算（默认 2560）
- [] 输出：最终视觉 token 数（int，不超过 max\_tokens）
- [] 超预算时按比例缩小分辨率重算（token 预算控制）

步骤提示：

```
`python
def dynamic_tokens(h, w, patch=14, compress=4, max_tokens=2560):
 """
```

Steps:

1. 计算 patch 数量 raw = ceil(h/patch) \* ceil(w/patch)
2. 应用压缩率 tokens = raw // compress（pixel shuffle ÷ 4）
3. 若 tokens > max\_tokens：h/w 各乘 0.8 取整后重算
4. 返回 token 数

```
"""
```

## TODO: Implement

```
pass
```

```
`
```

</details>

## 进阶与缺口（面试向：本课未深挖的高频考点）

- Q-Former / BLIP-2：在 ViT 与 LLM 之间加一个可学习的 Query Transformer（32 个 learnable query 通过 cross-attention 从 ViT 提特征）——token 压缩谱系的另一极（pixel shuffle 是"无参压缩"，Q-Former 是"可学习压缩"）；面试高频，答出"可学习 query 做信息瓶颈"即可及格。
- VLM 评估：MMMU（大学多学科推理）/ MME（14 子任务全景）/ MMBench / DocVQA、OCRBench（文档/文字类）——与 Part 8 07 章的评估学同构：固定题集、防污染、分类报分。
- VLM 幻觉：物体幻觉（图中没有却说有）是 VLM 特有病灶；评测用 POPE（对是否存在做二分探针）/ CHAIR（逐 token 统计幻觉物体）；成因与缓解（对比解码、RLHF-V）是当前热点。
- VLM 微调工具：LLaMA-Factory 原生支持 VLM 的 LoRA/SFT（--dataset 传多模态数据集即可）——Part 12 的工具链在多模态下几乎不变，这是"学一次用两处"的典型。
- 数据构造：caption 质量 > 数量（recaptioning 用强模型重写描述）；interleaved 图文交错数据；OCR/图表/文档类配比——多模态岗面试的数据题都绕不开这四点。

## 课后作业

[Assignment 15](../../assignments/assignment\_15/)

## 下一步

理解侧会"看"了，生成侧呢？Part 16 从 DDPM 的手写开始，走进扩散模型、文生图、图生图与视频生成——并且继续沿"特征对齐"主线深入。

[Part 16 图像/视频生成](../../Part16\_image\_video\_generation/tutorial/README.md)